

User manual for

Cutbank: A numerical model for river meandering with heterogeneous bank materials

Updated July 31, 2026

Ajay B. Limaye, Department of Environmental Sciences, University of Virginia

Acknowledgements: The code described in this manual originated in collaboration with Michael Lamb at Caltech. Thanks to Yuan Li, Doug Edmonds, Madison Douglas, and Sam Kodama for testing the code; and to Gabriela Birardi for code testing and contributions to the manual.

Table of Contents

1. Introduction.....	2
2. Modeling approach	2
3. Setting up the model environment	3
4. Quick start.....	5
5. Designing and running new simulations	12
a. Detailed instruction for Option 1(wrapper scripts).....	12
b. Detailed instructions for Option 2 (GUI).....	13
6. Code components.....	19
a. Table 1. Model input parameters	20
b. Table 2. Key model outputs	23
c. Table 3. Parameters for exporting additional grids and movies	24
d. Table 4. Function and scripts.....	25
7. References	29

1 Introduction

This user manual summarizes a code for modeling landscape evolution by a meandering river. Some preliminaries:

- This code is subject to the included GNU General Public License v3.0, which allows re-distribution and re-use of a licensed work on the condition that the creator is appropriately credited. Please observe the guidelines for additional license files that pertain to code written by others that is included in this archive. If you use or adapt this model for work that results in a publication, please acknowledge Cutbank in the text and cite one or more papers in the References at the end of this document as appropriate.
- This code has been tested for MATLAB R2025b on Windows (64-bit operating system); running on MacOSX and Linux may require minor changes. While MATLAB code is often forward-compatible, if you use a newer version of MATLAB some minor adjustments may be necessary.
- Throughout the manual, `Courier` font denotes MATLAB file paths, functions, scripts, and variables. The symbol `>>` indicates the MATLAB command line.
- See in-line comments within individual code files for details on implementation of the concepts summarized in this manual.
- If you encounter issues running the code or wish to submit constructive feedback, please contact Ajay B. Limaye (ajay@virginia.edu). This software is provided as-is with no performance guarantees.

This manual can be used in several ways:

- For an overview of the code design and its unique capabilities, see **modelling approach** (section 2).
- To **set up the model environment**, see section 3.
- To quickly explore the model capabilities and generate outputs, use the **quick start** (section 4). This section describes how a specific geomorphic process is represented in the model and shows results from example execution scripts included in this archive.
- For further information on **designing and running new simulations**, see section 5.
- For a deeper dive on the particulars, see **code structure** (section 6). This section includes a flow-chart that summarizes code execution, and a comprehensive list of code files and model input variables.

2 Modeling approach

This software implements and builds on the kinematic model for meandering river evolution developed by Howard & Knutson (1984). The original model describes the time evolution of a channel centerline (i.e., the line midway between the banks of a channel with fixed width). Neither flow nor sediment transport is explicitly modeled, although sediment transport is implicitly modeled in cases (e.g., point bar deposition). This implementation adds a periodic boundary condition; enables tracking bank materials, using either a grid-based or a vector-based framework for modeling the co-evolution of channel migration and bank-material-properties; and includes a glossary of functions for exporting movies and topographic data. Key components of the model are summarized below. Note that while the model contains many capabilities, only some are used in any single model run.

River meandering. The river channel is discretized using an ordered set of nodes that define the channel centerline and banks. At each time step, relative channel migration rates are calculated using a convolution of channel curvature weighted by distance upstream. Channel migration rates are adjusted based on the intrinsic properties of bank-materials and their spatial distribution in the model domain.

Change in elevation of the channel bed (vertical incision and aggradation). The channel longitudinal profile can be initialized as flat or with a constant longitudinal slope. Overall changes in channel bed elevation are directly imposed as independent variables. The model accommodates rates of bed elevation change that vary in time (steady vs. unsteady) and in space (uniform vs. non-uniform). For the non-uniform, incising case, local vertical incision rates are calculated using a slope-dependent erosion rule. Bed elevation change is negative for incision and positive for aggradation.

Bank-material tracking. The model can run with or without interaction between channel lateral erosion and bank-material properties. This interaction can be modeled using either vector-based bank-material tracking or using traditional, grid-based bank-material tracking. The grid-based approach is simpler for many applications, but model results can be highly sensitive to grid resolution in cases with large contrasts in erodibility of the bank materials (e.g., between sediment and bedrock). For examples see Figures 3, 5 and 9 in Limaye & Lamb (2013).

Visualization. Model results are visualized using post-processing scripts, which provide options to generate movies, movie frames as image files, and grids of geospatial data. For simulations of erosion, built-in functions map river terraces (for details, see Limaye and Lamb (2016), section 3.2.1).

3 Setting up the model environment

Get ready to run the model in three simple steps.

Step 1: Download MATLAB. The model runs on Linux, Mac, and PC and has been tested using MATLAB R2024b. This code uses the following MATLAB toolboxes:

- Statistics and Machine Learning Toolbox
- Image Processing Toolbox
- Mapping Toolbox
- Curve Fitting Toolbox
- Simulink Verification and Validation

To view the list of installed toolboxes in your version of MATLAB, type `ver` at the MATLAB command prompt. For more information on toolboxes, see [MATLAB documentation](#).

Step 2: Get oriented to the directory structure. The software includes many scripts and specialized functions. A good starting point is to familiarize yourself with the directory structure. Within the main directory are the following four subdirectories:

wrappers: Contains example scripts to define model input variables and execute the model for specific scenarios.

functions: Contains specialized functions that work together to execute each model run. Subdirectories include:

3rdParty: Code written by others

gridded: Functions specific to model runs that track bank-material properties using grids

GUI: Codes that generate graphical-user interfaces to specify input parameters and run the model

terrace_mapping: Includes codes used to map river terraces in some model runs.

utilities: Contains additional scripts that are useful for preparing model input files and visualizing the data structures that track bank materials.

output: Files generated in each model run are automatically placed in this directory. A subdirectory is created for each model run during execution.

Step 3: Compile code. Some of the code relies on `PolygonClip`, a function written in C that must be compiled prior to running the model for the first time. To do this, start from the base directory and navigate to the directly to the folder for `PolygonClip` as follows. From the MATLAB command line, enter:

```
>> cd .\functions\3rdParty\PolygonClip
```

Next run the script to compile the software by running `compile_PolygonClip.m` from the command line:

```
>> compile_PolygonClip
```

The script looks for an existing C compiler; if it does not find one, the command window should provide information for how to download and install one. Once the compiler is set up the command window should display something like this:

```
MEX configured to use 'MinGW64 Compiler (C)' for C language compilation.  
Building with 'MinGW64 Compiler (C)'.  
MEX completed successfully.
```

To check the path to executable file, type this in the MATLAB command window:
`which PolygonClip`

On a Windows PC with a 64-bit architecture, file path returned should end like this:

```
.\functions\3rdParty\PolygonClip\PolygonClip.mexw64
```

The file extension will differ for different operating systems, including Mac/Linux.

Once this step is complete, you are ready to run the model.

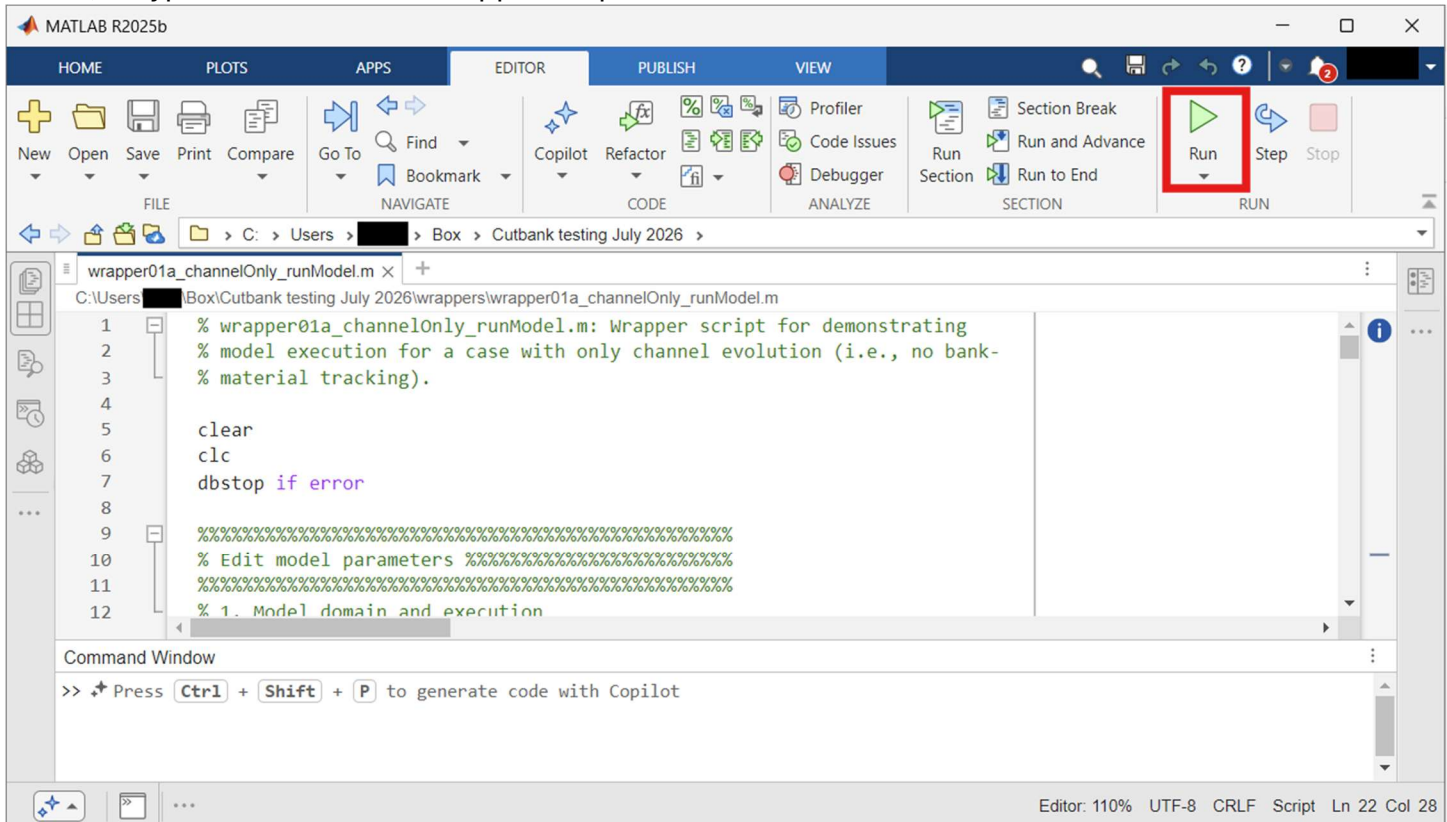
4 Quick-start

The fastest way to generate model outputs is to run with the example wrappers scripts (in the `wrappers` directory). Each example includes two scripts that specify parameters to (1) execute the model, and (2) generate visualizations from the model output. Each of the following examples illustrates a different model capability using the included example scripts. Note that your output may look slightly different because the examples are initialized with a random initial condition for the channel geometry.

Example 1: Channel evolution, without tracking surrounding topography or bank materials. This example isolates the sub-model for river meandering that underlies all other model scenarios. In this example a river 20 m wide and 1 m deep develops from a nearly straight initial planform geometry, with maximum a lateral erosion rate of 1 m/yr, over a period of 1000 years. The time step is set to 2.5 years. This example executes the following scripts in sequence.

- Execute the model: wrapper01a_channelOnly_runModel.m
- Export a movie from model output: wrapper01b_channelOnly_exportMovie.m

To execute the model script, set the current directory to wrappers and open wrapper01a_channelOnly_runModel.m. With the wrapper script open in the MATLAB Editor, click “Run”; or type the name of the wrapper script.



Alternatively, run the wrapper script at the MATLAB command line by typing its name (without the .m file extension) and hitting Enter. As the model runs, it episodically prints its progress to the command window (e.g., run1 50 percent complete).

```

15 domain.xExtentChannelWidths = 100; % Extent of model domain, in channel widths.
16 centerline_spacing_coeff = 1; % Coefficient that sets node spacing coefficient along channel centerline,
17 BMT = 'channel-only'; % 'channel-only' | 'vector-based' | 'grid-based'. Method for bank-material tracki
18 chkpt_spacing_coeff = false; % Vector-based BMT only: controls the spacing of checkpoints along a bank-m
19 bed_elev_chg_poly_addn_thresh = NaN; % Vector-based BMT only: cumulative vertical erosion before a new p
20
21 % 2. Channel geometry and kinematics
22 w = 20; % Channel width, m.
23 D = 1; % Channel depth, m
24 init_centerline_type = 'straight'; % 'straight' | 'evolved' | 'sinusoidal' | 'custom'
25 init_centerline_file = ''; % Name of file that stores initial coordinates of channel centerline (optiona

```

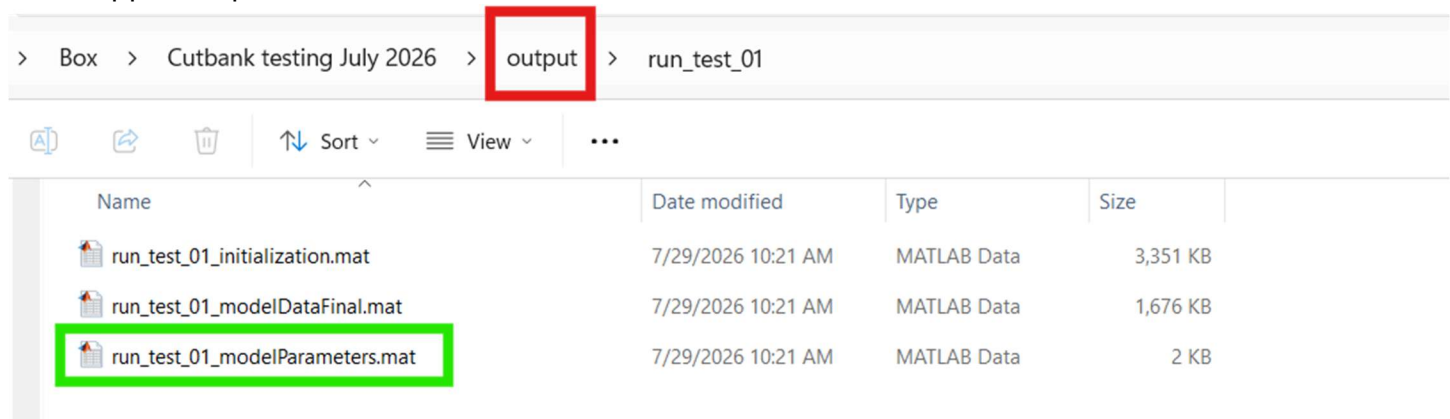
Command Window

```

Run test_01, started initialization
Run test_01, completed initialization
Finished run test_01
>> Press Ctrl + Shift + P to generate code with Copilot

```

You can also monitor progress by checking the output files in the output directory (`outputDir`) defined in the wrapper script.



All output files include the run name defined by the variable `trial` (run01 for this case). At the start of the model run, a file with the name “initialization” is created (`run01_initialization.mat`), and another file with the word “final” is created at the end of the model run (`run1_modelDataFinal.mat`).

Next, export a movie using `wrapper01b_channelOnly_exportMovie.m`. The previous step generates a time-series of vector data for the channel position. This step converts that vector data to a movie. As with the model run, a wrapper script is used to define parameters for data export.

```
wrapper01a_channelOnly_runModel.m x wrapper01b_channelOnly_exportMovie.m x +
C:\Users\Box\Cutbank testing July 2026\wrappers\wrapper01b_channelOnly_exportMovie.m
1 % wrapper01b_channelOnly_exportMovie.m: Wrapper script demonstrating movie
2 % file export for the model run of a channel only (i.e., no bank-material tracking).
3
4 clear
5 clc
6 dbstop if error
7
8 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
9 % Edit export parameters %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
10 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
11
12 % 1. Source files
13 modelParameterFile = 'run_test_01_modelParameters.mat'; % Full path to parameter file
14
```

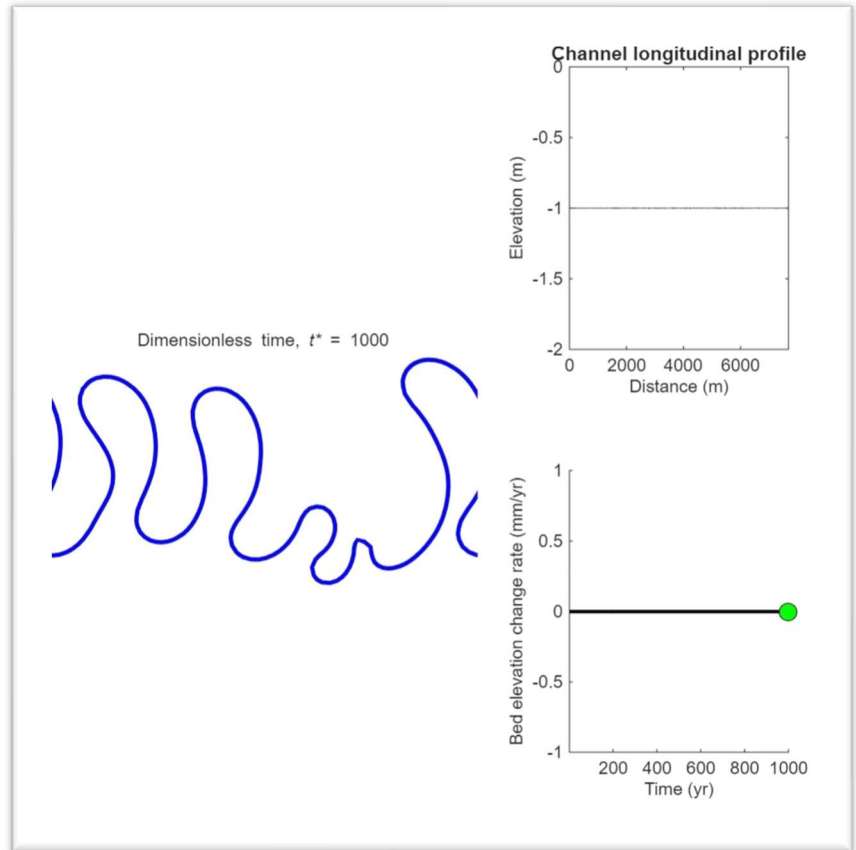
As with the model run, the script for file export will also progress to the command window as it runs. This example wrapper script exports a .mp4 file; edit the wrapper to output an animated GIF if desired. Other outputs are a subdirectory `movieFrames` that includes static movie frames, and a .mat file that saves the parameters for movie export. You can view these files in File Explorer on Windows (or Finder on Mac).

output > run_test_01 >

ew ...

Name	Date modified	Type
movieFrames	7/29/2026 10:42 AM	File folder
run_test_01_exportParameters.mat	7/29/2026 10:42 AM	MATLAB Data
run_test_01_initialization.mat	7/29/2026 10:21 AM	MATLAB Data
run_test_01_modelDataFinal.mat	7/29/2026 10:21 AM	MATLAB Data
run_test_01_modelParameters.mat	7/29/2026 10:21 AM	MATLAB Data
test_01_movie.mp4	7/29/2026 10:42 AM	MP4 File

A static image in the `movieFrames` directory captures a snapshot of the movie (example shown to the right). This example shows the channel flowing from left to right (upper left), where the indicated dimensionless time is equal to the product of run time and lateral erosion rate divided by channel width. The channel longitudinal profile (upper right) plots the elevation of the channel bed versus downstream distance; in this example, the channel is set to have zero slope; see Limaye and Lamb (2014) for further discussion. The time-series of the rate of change of the elevation of the channel bed (lower right) includes a slider (green) to indicate the current time. In this example the channel bed is neither incising vertically nor aggrading, so this rate is zero.



Example 2: Erosion of a bedrock river valley, using a vector-based approach to tracking bank materials.

- Execute the model: `wrapper02a_bedrockValley_vectorBased_runModel.m`
- Export topography grids and movies from model output:
`wrapper02b_bedrockValley_vectorBased_exportGridsMovie.m`

This example incorporates a longer run time (10,000 years), vertical incision (1 mm/yr), and bedrock that is 10x more resistant to erosion than the sediment. For a vertically incising channel can erode bedrock on the channel bed and eroding bank, and deposit sediment on the other bank. A sediment-filled zone, bounded by bedrock and one channel depth in thickness (i.e., the initial alluvial belt), is specified in this example. The initial alluvial belt extends the full length of the model domain in the mean downstream direction and an arbitrary width in the mean cross-stream direction. The variable `voidpoly` records the areas of the model domain that has not been visited by the channel. The variable `Data` stores several the dimensions of polygons that record bank-material properties. This application is described in section 5.3 of Limaye & Lamb (2013) and applied in Limaye & Lamb (2014, 2016).

Execute the above scripts in sequence.

Command Window Run test_02, started initialization Run test_02, completed initialization Run test_02, 5 percent complete Saving data for run test_02, t=1000 yr Run test_02, 10 percent complete Run test_02, 15 percent complete Saving data for run test_02, t=2000 yr Run test_02, 20 percent complete Run test_02, 25 percent complete Saving data for run test_02, t=3000 yr Run test_02, 30 percent complete Run test_02, 35 percent complete Saving data for run test_02, t=4000 yr Run test_02, 40 percent complete Run test_02, 45 percent complete Saving data for run test_02, t=5000 yr Run test_02, 50 percent complete Run test_02, 55 percent complete Saving data for run test_02, t=6000 yr Run test_02, 60 percent complete Run test_02, 65 percent complete Saving data for run test_02, t=7000 yr Run test_02, 70 percent complete Run test_02, 75 percent complete Saving data for run test_02, t=8000 yr Run test_02, 80 percent complete Run test_02, 85 percent complete Saving data for run test_02, t=9000 yr Run test_02, 90 percent complete Run test_02, 95 percent complete Saving data for run test_02, t=10000 yr Run test_02, 100 percent complete Finished run test_02	> output > run_test_02 > Search run_test_02 Q View > ... Details <table><thead><tr><th>Name</th><th>Date modified</th><th>Type</th></tr></thead><tbody><tr><td> run_test_02_initialization.mat</td><td>7/29/2026 12:11 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_1000.mat</td><td>7/29/2026 12:13 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_2000.mat</td><td>7/29/2026 12:14 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_3000.mat</td><td>7/29/2026 12:16 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_4000.mat</td><td>7/29/2026 12:19 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_5000.mat</td><td>7/29/2026 12:22 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_6000.mat</td><td>7/29/2026 12:25 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_7000.mat</td><td>7/29/2026 12:28 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_8000.mat</td><td>7/29/2026 12:32 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_9000.mat</td><td>7/29/2026 12:36 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelData_time_10000.mat</td><td>7/29/2026 12:42 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelDataFinal.mat</td><td>7/29/2026 12:42 PM</td><td>MATLAB Data</td></tr><tr><td> run_test_02_modelParameters.mat</td><td>7/29/2026 12:11 PM</td><td>MATLAB Data</td></tr></tbody></table>	Name	Date modified	Type	 run_test_02_initialization.mat	7/29/2026 12:11 PM	MATLAB Data	 run_test_02_modelData_time_1000.mat	7/29/2026 12:13 PM	MATLAB Data	 run_test_02_modelData_time_2000.mat	7/29/2026 12:14 PM	MATLAB Data	 run_test_02_modelData_time_3000.mat	7/29/2026 12:16 PM	MATLAB Data	 run_test_02_modelData_time_4000.mat	7/29/2026 12:19 PM	MATLAB Data	 run_test_02_modelData_time_5000.mat	7/29/2026 12:22 PM	MATLAB Data	 run_test_02_modelData_time_6000.mat	7/29/2026 12:25 PM	MATLAB Data	 run_test_02_modelData_time_7000.mat	7/29/2026 12:28 PM	MATLAB Data	 run_test_02_modelData_time_8000.mat	7/29/2026 12:32 PM	MATLAB Data	 run_test_02_modelData_time_9000.mat	7/29/2026 12:36 PM	MATLAB Data	 run_test_02_modelData_time_10000.mat	7/29/2026 12:42 PM	MATLAB Data	 run_test_02_modelDataFinal.mat	7/29/2026 12:42 PM	MATLAB Data	 run_test_02_modelParameters.mat	7/29/2026 12:11 PM	MATLAB Data
Name	Date modified	Type																																									
 run_test_02_initialization.mat	7/29/2026 12:11 PM	MATLAB Data																																									
 run_test_02_modelData_time_1000.mat	7/29/2026 12:13 PM	MATLAB Data																																									
 run_test_02_modelData_time_2000.mat	7/29/2026 12:14 PM	MATLAB Data																																									
 run_test_02_modelData_time_3000.mat	7/29/2026 12:16 PM	MATLAB Data																																									
 run_test_02_modelData_time_4000.mat	7/29/2026 12:19 PM	MATLAB Data																																									
 run_test_02_modelData_time_5000.mat	7/29/2026 12:22 PM	MATLAB Data																																									
 run_test_02_modelData_time_6000.mat	7/29/2026 12:25 PM	MATLAB Data																																									
 run_test_02_modelData_time_7000.mat	7/29/2026 12:28 PM	MATLAB Data																																									
 run_test_02_modelData_time_8000.mat	7/29/2026 12:32 PM	MATLAB Data																																									
 run_test_02_modelData_time_9000.mat	7/29/2026 12:36 PM	MATLAB Data																																									
 run_test_02_modelData_time_10000.mat	7/29/2026 12:42 PM	MATLAB Data																																									
 run_test_02_modelDataFinal.mat	7/29/2026 12:42 PM	MATLAB Data																																									
 run_test_02_modelParameters.mat	7/29/2026 12:11 PM	MATLAB Data																																									
wrapper_02a... takes longer to run than for Example 1 because the simulation time is longer and tracking for bank-material properties is enabled. The command window provides updates as new files are saved.	Output data files populate in the output directory, within a folder named for the model run.																																										

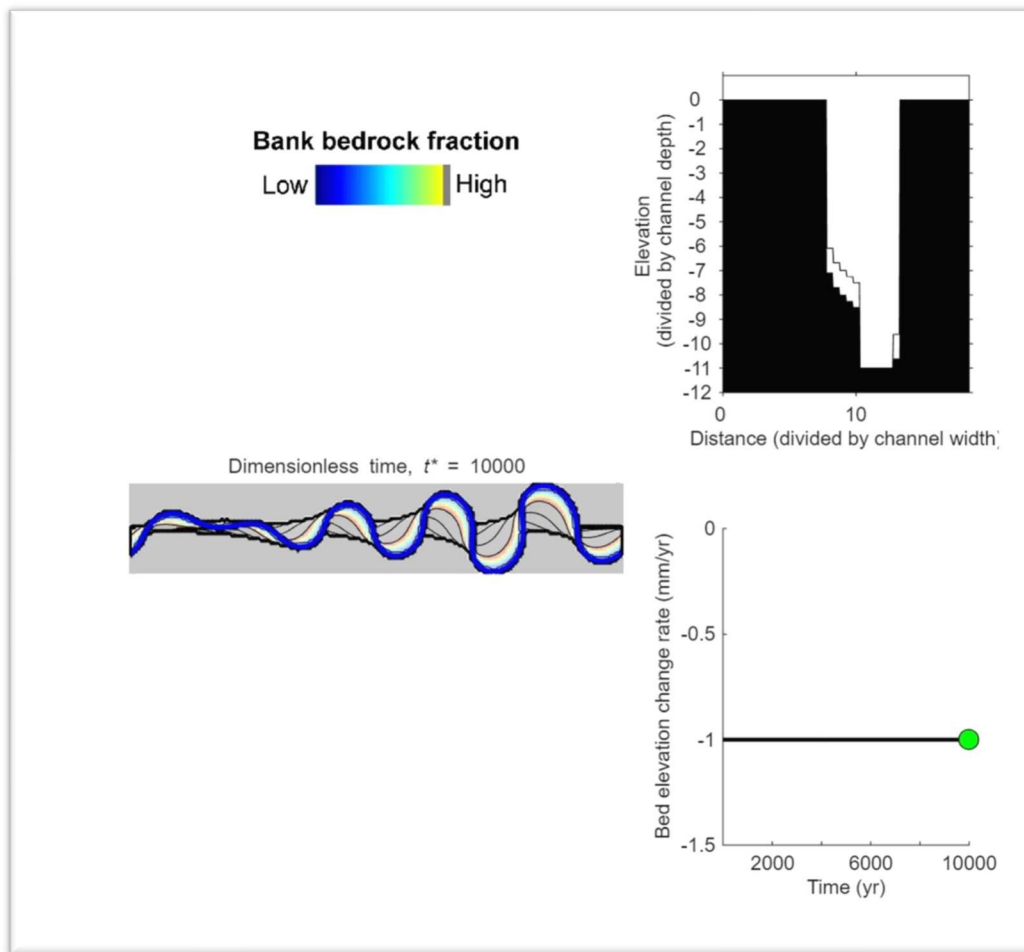
> output > run_test_02 > Search run_test_02

View ▾ ...

Name	Date modified	Type
grids	7/29/2026 12:53 PM	File folder
movieFrames	7/29/2026 12:53 PM	File folder
run_test_02_exportParameters.mat	7/29/2026 12:52 PM	MATLAB Data
run_test_02_initialization.mat	7/29/2026 12:11 PM	MATLAB Data
run_test_02_modelData_time_1000.mat	7/29/2026 12:13 PM	MATLAB Data
run_test_02_modelData_time_2000.mat	7/29/2026 12:14 PM	MATLAB Data
run_test_02_modelData_time_3000.mat	7/29/2026 12:16 PM	MATLAB Data
run_test_02_modelData_time_4000.mat	7/29/2026 12:19 PM	MATLAB Data
run_test_02_modelData_time_5000.mat	7/29/2026 12:22 PM	MATLAB Data
run_test_02_modelData_time_6000.mat	7/29/2026 12:25 PM	MATLAB Data
run_test_02_modelData_time_7000.mat	7/29/2026 12:28 PM	MATLAB Data
run_test_02_modelData_time_8000.mat	7/29/2026 12:32 PM	MATLAB Data
run_test_02_modelData_time_9000.mat	7/29/2026 12:36 PM	MATLAB Data
run_test_02_modelData_time_10000.mat	7/29/2026 12:42 PM	MATLAB Data
run_test_02_modelDataFinal.mat	7/29/2026 12:42 PM	MATLAB Data
run_test_02_modelParameters.mat	7/29/2026 12:11 PM	MATLAB Data
test_02_movie.mp4	7/29/2026 12:53 PM	MP4 File

After running `wrapper_02b...`, output files for visualization populate as before. For this example, however, an additional subdirectory `grids` includes gridded data files generated from post-processing. (These gridded files were not generated during the original model run that used the vector-based approach to bank-material tracking; the grids are generated to support visualization after).

The gridded data represent the properties of the landscape (i.e., topography and bank-material properties) and are exported in the data structure `grid`, with fields `nrows` (number of grid rows), `ncols` (number of grid columns), `x` (grid x-coordinates, in meters), `y` (grid y-coordinates, in meters), `z` (surface elevation, in meters), and `z_bedrock` (bedrock elevation, in meters), and `t` (time of last surface scour by the channel, in years).



The bank materials are composed of sediment and bedrock. In this example the fraction of bedrock in the bank materials is indicated with a color scale in the movie output (`test_02_movie.mp4`) at far left. Cross-section (upper right) shows bedrock in black and sediment in white, and corresponds to a transect across the valley, positioned halfway across model domain.

- Execute the model: `wrapper03a_bedrockValley_gridBased_runModel.m`
- Export topography grids and movies from model output:
`wrapper03b_bedrockValley_gridBased_exportGridsMovie.m`

Figure 10 illustrates the evolution of a meandering river. The top panel shows a map of the river bed at dimensionless time $t^* = 10000$, with a color scale for relative terrace elevation from Low (green) to High (yellow). The middle panel shows a cross-section of the river bed at $t^* = 10000$, with elevation (divided by channel depth) on the y-axis and distance (divided by channel width) on the x-axis. The bottom panel shows the bed elevation change rate (mm/yr) versus time (yr), with a constant rate of -1 mm/yr from 0 to 10000 years.

Further model capabilities. The example wrappers can be modified to treat additional scenarios for aggrading channels. These scenarios include:

- **Channel aggradation and overbank deposition.** In this scenario, the channel aggrades at a constant rate and the surrounding floodplain aggrades with two components (one spatially uniform, one with an exponential dependence on distance from the channel). During the model run, bank height is reconstructed at each point on the eroding bank, and channel lateral erosion rate is adjusted to vary inversely with bank height. This application is described in section 5.4 of Limaye & Lamb (2013). Key variables for this scenario are `bed_elev_chg_rate`, which should be set positive for aggradation; and the data structure `overbank_deposition`, which specifies the parameters of the overbank deposition model. See Table 1 for further details.
- **Modification of lateral migration rates by resistant floodplain sediments.** Through neck cutoffs, meandering rivers can generate low areas that preferentially trap fine sediment. Such deposits can locally inhibit channel lateral erosion. In the model, a separate erodibility coefficient for lateral erosion can be applied to areas abandoned by neck cutoff. This application is described in section 5.1 of Limaye & Lamb (2013). Key variables for this scenario are `overbank_deposition.modify_lateral_erosion_resistant_oxbows`, which enables the process; and `overbank_deposition.oxbow_erode_coeff`, which specifies the erodibility contrast for oxbow-filling sediments (Table 1).
- **Effects of bank strength on channel-body geometry for aggrading channels.** Aggrading channels form packages of coarse sediment, or channel bodies. This model scenario is similar to the previous scenario, but with an aggrading channel (`bed_elev_chg_rate > 0`). Therefore, resistant sediments are tracked in three dimensions to account for deposit geometry. This application is described in section 5.2 of Limaye & Lamb (2013).
- **Tracking age of eroded sediment.** Channel migration is a key control on sediment residence time in floodplains. Model scenarios using grid-based material tracking (using `meander_gridded.m`) to track the timing of surface scour by the channel (in the grid `dem.t`) and the age of sediment eroded by the channel in each timestep (in the cell array `dem.eroded_sediment_time_since_emplacement`). These variables are saved to data files from model output (Table 2). This application is described in Torres et al. (2017).

5 Designing and running new simulations

To design and run new simulations, there are two options:

- **Option 1: Modify an existing example script** from the `wrappers` directory, as used in section 4. This option is ideal for users familiar with MATLAB syntax and data types, and who wish to access all model and file export capabilities. Best practices are to rename the wrapper script; and within the script, to change the variable `trial`, which is used as the base name for output files.
- **Option 2: Use the graphical user interface (GUI).** This option is ideal for users who are new to MATLAB and are willing to sacrifice some control over model parameters. (Some model capabilities are disabled to simplify the interface, and some file export capabilities are disabled for compatibility with the GUI). Directions for using the GUI follow.

Detailed instructions for Option 1 (wrapper scripts). Create a new wrapper script by copying from an example script and changing the parameter values. **Important:** set a unique value for the variable `trial` should be used for all model runs to prevent overwriting data from previous runs. Precise syntax is

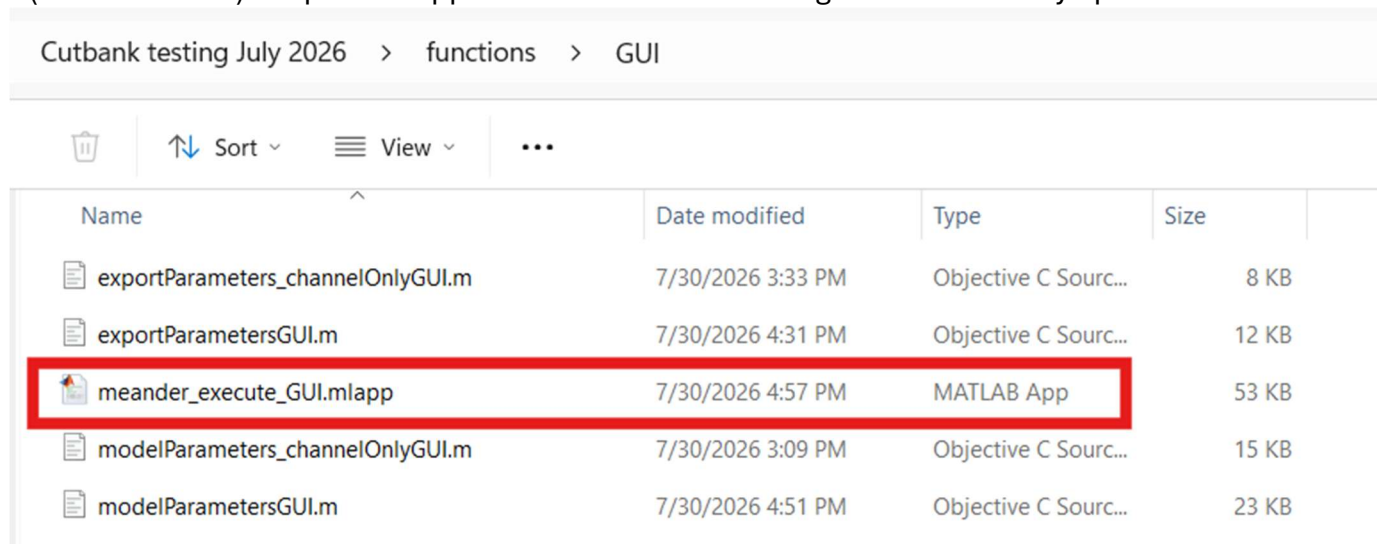
necessary for defining variables; for guidance, see comments included in the example scripts, which is duplicated in section 5 (Table 1).

Tips:

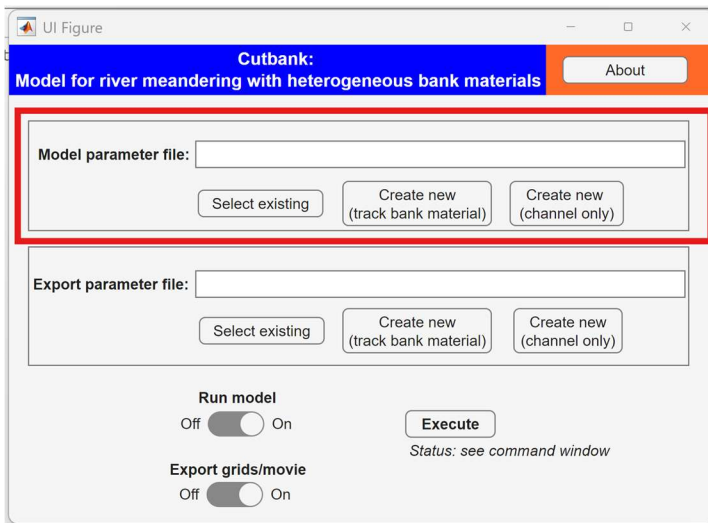
- Several functions include error handling in certain locations. If your model run triggers one of these error checks, it will save a file the output directory formatted as `run_name_error_data.mat`. The variable `err` describes the error and the name of the function that threw the error. This file also preserves variables at the occurrence of the error, which are helpful to diagnose the problem.
- To further customize movies, exported still frames can be used in software packages such as iMovie (Mac) or Movie Maker (PC). For Movie Maker, import the images, set the duration of each frame, and add overlay text as desired. Save the movie file, then export using the desired quality and format.
- MPEG-4 is a flexible export format compatible with both Mac and PC. Additional software may be useful to further reduce movie file size, for example, FFmpeg (<https://ffmpeg.org/>).

Detailed instructions for Option 2 (GUI).

Step 1: Open the main GUI. Navigate to the directory `/functions/GUI` and double-click on MATLAB App file (outlined in red) to open the app. MATLAB will start running if it is not already open.



The Cutbank app will open in a new window.

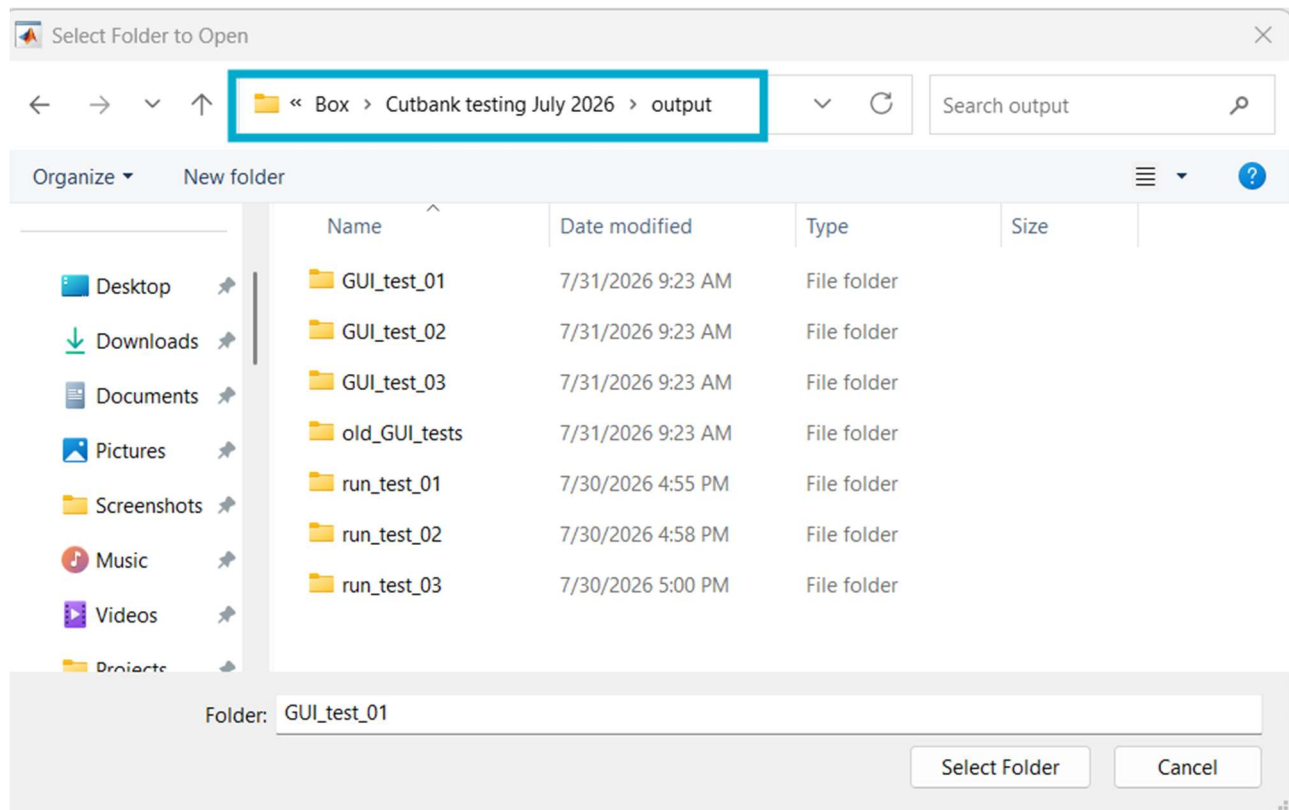


Step 2: define the model parameter file. The top box (outlined in red) is used to select an existing model parameter file or create a new one. To choose an existing file, click the **Select existing** button, and a pop-up window will allow you to search within the current file structure for the file. Once the file is chosen, its address will appear in the blank box above the buttons.

To create a new model parameter file, click one of the **Create new** buttons. (Click the “track bank material” version for a model run that will track bank materials; use the “channel only” button for a model run that only models the

channel and not the surrounding landscape). Clicking either of the corresponding buttons opens pop-up window for you to select model parameters. This is the pop-up window for the “track bank material” option:

Every parameter is required unless **(optional)** is written beside the name, each parameter is required. Default values populate automatically. See Table 1 (section 5) for detailed information on each parameter. Click in the box for **Output directory** (outlined in pink above) to launch a directory search; the \outputs is recommended, and you can create new subdirectory for this model run.



After selecting the output directory and finalizing parameters, Click **OK** to save the model parameter file. The window will close. The model parameter file will be created within the chosen folder; this can be confirmed by checking the folder, and within the Command Window in MATLAB, a message will be generated:

```
Command Window
Default parameters assigned for vector-based bank material tracking
Wrote C:\Users\gsbir\Box\Cutbank testing July 2026\output\GUI_test_01\run_test_inputs.mat to C:\Users\gsbir\Box\Cutbank testing July 20
```

The process is similar for the “channel only” option, except there are fewer parameters to select. Again, the **Output directory** (outlined in pink) is required; the app will throw an error specifying that the output directory is missing.

Meandering model inputs (Channel-only)

1. Model domain and execution

Run time (yr)

Time increment (yr)

x-direction domain extent (channel widths)

Centerline node spacing (channel widths)

Bank material tracking

2. Channel geometry and kinematics

Channel width, w_c (m)

Channel depth, h_c (m)

Initial channel centerline geometry

Initial centerline file (optional)

Initial surface elevation outside channel (m)

Initial slope

Sediment lateral erosion rate, E_{Ls} (m/yr)

Vertical incision style

Rate of bed elevation change, dz/dt (m/yr)

Time series of vertical incision rate (optional)

Limit channel displacement in one time step to w_c ☒ yes ☐ no

3. Meandering model constants

Friction coefficient, C_f

k

Ω

Γ

ϵ

4. Physical constants

Gravitational acceleration, g (m/s²)

Water density, ρ (kg/m³)

5. Input files

Existing input parameter file (optional)

Path of file to restart simulation from (optional)

6. Output files

Base name for file output

Output directory

File output interval (yr)

OK Cancel

Step 3: Define the export parameter file. Similarly to the model parameter file, you have three options: **Select existing file**, **Create new (track bank material)**, or **Create new (channel only)**. Your choice must be consistent with the previous step.

This is the pop-up window for the **Create new (track bank material)** option:

The file locations are automatically generated based on the currently selected model parameters file (see addresses outlined in pink above). This window allows you to choose the format for the output of the model. (For more information on parameters see Table 1 in section 5). After setting parameters, click **OK** to generate an export parameter file; a confirmation prints to the command window (highlighted in green below).

```
Command Window
Wrote C:\Users\gsbir\Box\Cutbank testing July 2026\output\GUI_test_01\run_test_inputs.mat to C:\Users\gsbir\Box\Cutbank testing July 20
Wrote C:\Users\gsbir\Box\Cutbank testing July 2026\output\GUI_test_01\run_test_exportParameters.mat to C:\Users\gsbir\Box\Cutbank testi
```

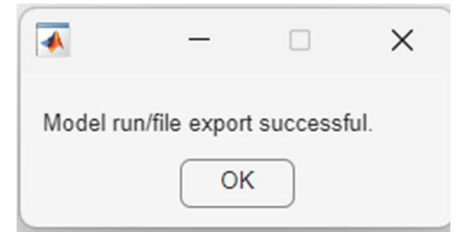
For the **Create new (channel only)** option, the pop-up window has fewer parameters. The file locations will auto-populate (pink below):

Step 4: Specify whether to run the model and/or export grids/movie. Specify these options by toggling the buttons (highlighted in red). In some cases it may be advantageous to just re-run the model without generating further outputs, or to re-run movie export without running the model. By default, both options are set to **On**.

Once you are ready, click **Execute** to run Cutbank. As the model runs, status updates appear in the MATLAB Command Window.

```
Command Window
Run test, 5 percent complete
Run test, 10 percent complete
Run test, 15 percent complete
Run test, 20 percent complete
Run test, 25 percent complete
Run test, 30 percent complete
Run test, 35 percent complete
Run test, 40 percent complete
Run test, 45 percent complete
Run test, 50 percent complete
Run test, 55 percent complete
Run test, 60 percent complete
Run test, 65 percent complete
Run test, 70 percent complete
Run test, 75 percent complete
Run test, 80 percent complete
Run test, 85 percent complete
Run test, 90 percent complete
Run test, 95 percent complete
Run test, 100 percent complete
Finished movie export for run test
```

If the model and/or file export runs successfully, this dialog box will appear.



Check the file output directory you specified in Step 2 for all the files generated.

6 Code components

This section lists model parameters (Table 1), the key model outputs (Table 2), parameters that can specify export of additional grids and movies (Table 3), and all functions and scripts (Table 4). Tables begin on the next page.

Table 1. Model input parameters.

Not all parameters are used for all model runs; for example, model runs that treat channel migration only do not use parameters that involve tracking bank materials. Set unused parameters to `false` (logical data types), `NaN` (double data types), or `' '` (i.e., an empty string, for string data types).

Name	Data type	Description
<i>Model domain and execution</i>		
<code>t_max</code>	double	Run time (yr)
<code>t_increment</code>	double	Time increment (yr)
<code>domain.xExtentChannelWidths</code>	double	Downstream extent of model domain, in channel widths.
<code>centerline_spacing_coeff</code>	double	Spacing of nodes along channel centerline, in channel widths.
BMT	string	Mode of bank-material tracking. Specify 'vector-based' or 'grid-based' to track bank materials using either of those approaches, or 'channel-only' to skip modeling of bank materials.
<i>Vector-based parameters</i>		
<code>chkpt_spacing_coeff</code>	double	Spacing of checkpoints along a bank-material inspection vector (m).
<code>bed_elev_chg_poly_addn_thresh</code>	double	Cumulative change in the absolute value of the channel bed elevation before a new polygon for tracking bank materials is created (m)
<i>Grid-based parameters</i>		
<code>cell_width</code>	double	Width of a grid cell (m)
<i>Channel geometry and kinematics</i>		
<code>w</code>	double	Channel width (m)
<code>D</code>	double	Channel depth (m)
<code>init_centerline_type</code>	string	Specifies initial channel planform shape. 'straight' for a straight channel with small-scale white noise, 'evolved' for a channel evolved with the meandering model, 'sinusoidal' for a channel in the shape of a sine wave, or 'custom' to use a specified file with custom coordinates.
<code>init_centerline_file</code>	string	Name of file that stores initial coordinates of channel centerline (optional).
<code>init_plane_max_elev</code>	double	Elevation of initial surface outside of channel (m)
<code>init_channel_slope</code>	double	Initial channel slope (dimensionless)
<code>k_erode_sediment</code>	double	Rate coefficient for lateral erosion in sediment banks. Yields the maximum specified erosion rate in m/yr.
<code>vertical_incision_style</code>	string	Style of vertical incision. Choose 'flat_steady' for a channel with no slope and

		constant vertical incision rate; 'flat_unsteady' for a channel with no slope and an unsteady vertical incision rate; 'sloping_steady' for a channel with a slope and constant vertical incision rate; or 'shear_stress' for a channel with nonuniform vertical erosion rates set by the local slope at each channel centerline node.
bed_elev_chg_rate	double	Rate of channel bed elevation change (m/yr). Negative for incision, positive for aggradation.
vertical_file	string	Name of file for importing a predefined vertical incision rate time-series (optional)
Geomorphic processes		
Bedrock river valleys		
bedrock_erosion. modify_lateral_erosion _bedrock	logical	Flag for modifying lateral erosion rates due to bedrock (true or false).
k_erode_bedrock	double	Bedrock lateral erosion rate coefficient. Yields the maximum specified erosion rate in m/yr.
unconfined_alluvial _belt_width	double	The width of the alluvial belt under unconfined conditions.
init_alluv_width_coeff	double	The imposed alluvial belt width, as a fraction of the previous variable.
init_Ev_pulse	logical	Flag for starting the model run with a pulse of vertical incision (true or false).
pulse_coeff	double	Depth of initial channel incision, in channel depths (optional).
Floodplains		
overbank_deposition. modify_lateral_erosion _bank_height	logical	Flag for varying lateral erosion rate inversely with bank height (true or false).
overbank_deposition. modify_lateral_erosion_resistant_oxbows	logical	Flag for changing the lateral erosion rate for oxbows (true or false).
overbank_deposition. parameters. oxbow_erode_coeff	double	Fractional erodibility of oxbow-filling sediments relative to k_erode_sediment.
Parameters for the Howard (1992) overbank deposition model		
overbank_deposition. enable	logical	Flag for enabling overbank deposition.
overbank_deposition. parameters.nu	double	Rate of spatially uniform overbank deposition (m/yr).
overbank_deposition. parameters.mu_d	double	Maximum rate of space-dependent overbank deposition (m/yr).
overbank_deposition. parameters.lambda	double	e-folding length for exponential decay in space-dependent overbank deposition rate (m).
Meandering model constants		
Cf	double	Friction coefficient (dimensionless).

k	double	Coefficient for meandering model (dimensionless); k in Howard & Knutson (1984).
om	double	Coefficient for meandering model (dimensionless); Ω in Howard & Knutson (1984).
gam	double	Coefficient for meandering model (dimensionless); Γ in Howard & Knutson (1984).
epsilon	double	Coefficient for meandering model (dimensionless); ε in Howard & Knutson (1984).
Physical constants		
g	double	Gravitational acceleration (m/s^2).
rho	double	Density of water (kg/m^3).
Input files		
startfile	string	Path of file to restart simulation from (optional).
Output files		
trial	string	Name of model run; applied as the base name for output files.
save_interval	double	Time interval to save results (yr)

Table 2. Key model outputs.

Name	Data type	Description
Data	struct	Holds time-series data from the model run. Each element of the structure array corresponds to the same time step in the model of model time. For runs with vector-based bank-material tracking enabled, Data includes the fields <code>bdrk_topo_polygon</code>, which stores geometry information for each preserved polygon from that time step; <code>bounding_box</code>, which stores bounding boxes for each separate polygon in <code>bdrk_topo_polygon</code>; and <code>f_bedrock</code>, which is the fraction of the bank material that is composed of bedrock. For all model runs, Data also includes the following fields: <code>t</code>, time; <code>centerline</code>, an structure array that stores the coordinates (X,Y,Z) of each node on the channel centerline; <code>El_mean_median_max</code>, a 1x3 vector that records the mean, median, and maximum lateral erosion rate at that time step; <code>Ev_mean_median_max</code>, which records the same statistics for vertical incision rate; and <code>Rlprime</code>, which records the signed, relative migration rates for each node. For runs with grid-based bank-material tracking enabled, Data includes many of the same fields with timeseries related to the channel; the field <code>visited_area</code> tracks new area visited by the channel in each timestep.
dem	struct	For models with grid-based bank-material tracking enabled, this array includes spatial data in different fields. Examples include a grid of surface scour time (<code>dem.t</code>) and a cell array that tracks, for sediment eroded by the channel, the time since it was originally emplaced (<code>dem.eroded_sediment_time_since_emplacement</code>). For more on the other fields see comments in <code>meander_gridded.m</code>.
cutoff_interp_stats	struct	Holds time-series data for neck cutoffs and channel centerline interpolation from the model run. The fields are: <code>centerline_interp_log</code>, a vector that specifies for each time step whether the centerline was interpolated (<code>true</code> or <code>false</code>); <code>cutoff_log</code>, which does the same but for neck cutoffs that also alter the centerline; <code>n_cutoffs</code>, which records the number of neck cutoffs in each timestep; <code>cutoff_length</code>, a cell array in which each element gathers the lengths of channel removed individual cutoffs within each timestep; <code>nodes_removed_ranges</code>, which indicates the corresponding indices of nodes in the channel centerline that were removed in cutoffs; and <code>loop_coords</code>, which records the coordinates of the cutoff loops removed in Nx2 arrays, where the column 1 represents X-coordinates and column 2 represents Y-coordinates.
model_performance	struct	The field <code>seconds_per_iteration</code> records the elapsed time during the iteration proceeding file export. The field <code>run_time</code> records the total run time for the final exported data file as a 1x6 array with the format [years months days hours minutes seconds].

Table 3. Parameters for exporting additional grids and movies.

Name	Data type	Description
Source files		
modelParameterFile	string	Full path to model parameter file
Grid export		
exportParameters.grid.export	logical	Flag to export grids (true or false).
exportParameters.grid.cell_width	double	Grid spacing for exported grids.
exportParameters.grid.export_interval_yr	double	Time interval for grid export (yr).
Movie export		
exportParameters.movie.contour_interval_depths	double	Elevation contour interval, in channel depths
exportParameters.movie.xc_xtick_interval_widths	double	Plot x-tick interval for topographic cross-sections, in channel widths
exportParameters.movie.xc_ytick_interval_depths	double	Plot y-tick interval for topographic cross-sections, in channel depths
exportParameters.movie.image_width_cm	double	Image width for movie (cm)
exportParameters.movie.plot_f_bedrock	logical	Switch to plot bank-material bedrock fraction in mapview plots for bedrock river valleys (true or false).
exportParameters.movie.plot_terraces	logical	Switch to plot terraces in mapview plots for bedrock river valleys (true or false).
exportParameters.movie.format	string	File format for output movie (MPEG-4 or animated GIF)
exportParameters.movie.fps	double	Frames per second for movie.

Table 4. Function and scripts.

Function name	File path	Description
adjust_centerline_nodes.m	/functions	Moves the nodes in the channel centerline and maintains proper centerline geometry throughout the model run.
calc_increment.m	/functions	Calculates the (X,Y) increments for moving channel centerline nodes. Nodes are moved perpendicularly to the local channel azimuth in a direction determined by the sign of the variable <code>Rlprime</code> .
channel_margins.m	/functions	Creates channel banks from the channel centerline and width. Bank geometry is output for each bank (<code>leftBank</code> , <code>rightBank</code>) as well as the full channel boundary polygon (<code>channelFootprint</code>).
check_input_data_types.m	/functions	For model input parameters, converts string inputs to numeric and vice versa, as appropriate.
check_export_data_types.m	/functions	For file export parameters, converts string inputs to numeric and vice versa, as appropriate.
circle_only.m	/functions	Calculates radius of curvature between sets of three points by fitting a circle. Modified from <code>points2circle.m</code> by Jos van der Geest, available from the MATLAB File Exchange .
compile_mex.m	/functions	Compiles functions written in low-level programming languages that are invoked in MATLAB. This should be run prior to executing model runs.
create_centerline_geometry.m	/functions	Creates channel centerline geometry.
create_init_alluv_poly.m	/functions	Creates data structure <code>init_alluv_poly</code> with geometry information for the polygon that represents the initial alluvial belt.
create_polygon.m	/functions	Generates a polygon representing the channel planform extent. The channel centerline is copied periodically; a polygon is generated from the centerline; and the polygon is clipped to the model domain.
cutoff_interpolation.m	/functions	Locally adjusts the channel centerline to maintain node spacing within a defined range, and checks for neck cutoffs. The operations are performed iteratively until the channel planform meets these geometric criteria.
export_grids_movie.m	/functions	Generates grids and/or movies. Grids correspond to surface topography, bedrock topography, and surface scour time for model data using the time-series of channel planform traces and longitudinal profile, and geometry for the initial alluvial-belt, if any. Movie export generates a movie file and associated movie frames.

export_movie_channelOnly.m	/functions	Movie export for model runs with channel only (i.e., no bank-material tracking). Movie export generates a movie file and associated movie frames.
howard_knutson_periodic.m	/functions	Calculates channel lateral migration rates using an implementation of the Howard & Knutson (1984) meandering model.
locate_grid_intersections.m	/functions/grid dded	For grid-based bank-material tracking, identifies where the search vector extended from the bank intersects with grid cell edges, and returns the grid cell indices intersected and the length along the search vector that corresponds to each cell.
meander_execute.m	/functions	Function to execute meandering model (for vector-based, grid-based, or no bank material tracking) and to generate output grids and/or movies.
meander_gridded.m:	/functions/grid dded	Main function for modeling channel evolution within a grid-based framework for tracking bank materials.
GUI_exportParameters_channe lOnly.m:	/functions/GUI	Generates a graphical user interface (GUI) for file export parameters for runs with no bank-material tracking.
GUI_exportParameters.m	/functions/GUI	Generates a GUI for setting file export parameters for model runs with bank-material tracking.
GUI_modelParameters_channel Only.m	/functions/GUI	Generates a GUI for entering input parameters for the model for runs with no bank-material tracking.
GUI_modelParameters.m	/functions/GUI	Generates a GUI for entering input parameters for the model for runs with bank-material tracking.
GUI_meander_execute.m	/functions/GUI	Generates the main GUI that calls the other GUIs in this list and executes the model and grid/movie export.
meander_initialization.m	/functions	Runs initialization phase to determine dimensional erosion rate coefficients and channel centerline.
meander_vector.m	/functions	Main function for modeling channel evolution within a vector-based framework for tracking bank materials.
my_intersections.m	/functions	Finds intersections between two vectors. This function is lightly modified from intersections.m by Douglas M. Schwarz, available from the MATLAB File Exchange . Compared to the original function, a warning is turned off and "unique" has been removed from line 241.
my_unique.m	/functions	Faster version of MATLAB built-in function unique.m. Returns unique values of input row or column vectors.

myMinMax.m	/functions	Returns a 2-element vector with the minimum and maximum of the input vector. This is faster than separately calling the MATLAB built-in functions <code>min.m</code> and <code>max.m</code> .
nominal_migration_rate.m	/functions	Calculates nominal migration rate for each channel centerline node based on local curvature. Uses a periodic boundary condition to compute the curvature at the centerline endpoints.
replicate_centerline_periodic.m		Copies channel centerline upstream and downstream, which is used for the periodic boundary condition in the model.
map_terraces.m	/functions/terrace_mapping	Main function for mapping river terraces and calculating associated morphometrics. This function can be invoked for model cases with a vertically incising channel.
regressplane.m	/functions/terrace_mapping	Fits a plane to elevation data using linear regression. This code was developed by Kevin Lewis and Oded Aharonson at Caltech.
write_terrace_shapefile.m	/functions/terrace_mapping	Exports mapped terraces to a shapefile.
thresh_tool_pan_zoom_fp.m	/functions/3rd Party/thresh_tool	Tool for interactive selection of an image threshold value, applied in this case to selection of an estimated floodplain elevation. This function is based on <code>thresh_tool.m</code> 1.6.0.0 by Robert Bemis, from the MATLAB file Exchange .
thresh_tool_pan_zoom_elev.m	/functions/3rd Party/thresh_tool	Tool for interactive selection of an image threshold value, applied to elevation data. This function is based on <code>thresh_tool.m</code> 1.6.0.0 by Robert Bemis, from the MATLAB file Exchange .
inpoly.m	/functions/3rd Party/inpoly	Determines whether points lie within or outside of a polygon. Created by Darren Engwirda. Version 3.0.0.0 (released 12/19/2020). Available at the MATLAB File Exchange .
distance2curve.m	/functions/3rd Party/distance2curve	Calculates the distance from a set of points to a curve. Created by John D'Errico. Version 1.1.0.0 (released 2/27/2013). Available at the MATLAB File Exchange .
PolygonClip	/functions/3rd Party/PolygonClip	Calls the polygon clipper designed by Alan Murta, modified for MATLAB by Sebastian Holz (Geophysik, TU Berlin, Germany). Version 1.0.0.0 (released 10/4/2006). Available from the MATLAB File Exchange . More information about the original source code is available at Wikipedia . This function is written in C and is compiled as a MEX file during setup (see section 3).

hillshade.m	/functions/3rd Party/TopoToolbox/	Generates shaded relief map of gridded topography. Created by Wolfgang Schwanghart. Version 2.4 (released 9/22/2025). Available from the MATLAB File Exchange .
rgb.m	/functions/3rd Party/rgb	Generates a 1x3 vector with the red-green-blue values for an input color. Created by Kristján Jónasson, Dept. of Computer Science, University of Iceland. Version 1.1.0.0 (released 7/5/2009). Originally downloaded from the MATLAB File Exchange .
inpolyhedron.m	/functions/3rd Party/inpolyhedron	Determines whether points lie within a polyhedron defined by triangulated facets. Created by Sven. Version 3.3.0.0 (released 11/12/2015). Available at the MATLAB file Exchange .
inputsdlg.m	/functions/3rd Party/inputsdlg	Generates an input dialog box. Created by Takshi Ikuma with contributors Andreas Greuer, Luke Resiner, and Florian Hatz. Version 2.3.2.0 (released 6/30/2015). Available from the MATLAB File Exchange .
v2struct	/functions/3rd Party/v2struct	Packs variables in a structure array or unpacks them from a structure array. Created by Adi Navve. Version 1.14 (released 11/20/2014). Available from the MATLAB File Exchange .
upslope_area (software package)	/functions/3rd Party/upslope	Contains functions for computing and visualizing upslope area in a DEM. Created by Steven. L. Eddins, a developer at MathWorks, Inc. Version 1.3.0.1 (released 9/1/2016). Available from the MATLAB File Exchange .
createBedElevationChangeTimeseries.m	/utilities	Script demonstrating how to format an input timeseries of channel bed elevation change (erosion or aggradation) for compatibility with the model.
plotGrids_example.m	/utilities	Demonstrates how to plot gridded data files.
plotBankMaterialPolygons_example.m	/utilities	Example script to plot polygons used for vector-based bank-material tracking.

References

- Howard, A. D. (1992). Modeling channel migration and floodplain sedimentation in meandering streams. In P. A. Carling & G. E. Petts (Eds.), *Lowland floodplain rivers: Geomorphological perspectives* (pp. 1–41). John Wiley and Sons, Ltd.
- Howard, A. D., & Knutson, T. R. (1984). Sufficient conditions for river meandering—A simulation approach. *Water Resources Research*, 20(11), 1659–1667. <https://doi.org/10.1029/WR020i011p01659>
- Limaye, A. B. S., & Lamb, M. P. (2013). A vector-based method for bank-material tracking in coupled models of meandering and landscape evolution. *Journal of Geophysical Research: Earth Surface*, 118(4), 2421–2437. <https://doi.org/10.1002/2013JF002854>
- Limaye, A. B. S., & Lamb, M. P. (2014). Numerical simulations of bedrock valley evolution by meandering rivers with variable bank material. *Journal of Geophysical Research: Earth Surface*, 119(4), 927–950. <https://doi.org/10.1002/2013JF002997>
- Limaye, A. B. S., & Lamb, M. P. (2016). Numerical model predictions of autogenic fluvial terraces and comparison to climate change expectations. *Journal of Geophysical Research: Earth Surface*, 121(3), 512–544. <https://doi.org/10.1002/2014JF003392>
- Schwanghart, W., & Scherler, D. (2014). Short Communication: TopoToolbox 2 – MATLAB-based software for topographic analysis and modeling in Earth surface sciences. *Earth Surface Dynamics*, 2(1), 1–7. <https://doi.org/10.5194/esurf-2-1-2014>
- Torres, M. A., Limaye, A. B., Ganti, V., Lamb, M. P., West, A. J., & Fischer, W. W. (2017). Model predictions of long-lived storage of organic carbon in river deposits. *Earth Surface Dynamics*, 5(4), 711–730. <https://doi.org/10.5194/esurf-5-711-2017>